Guide démo live — Demo 1 (très pédagogique)

Public : client / audience non experte
Durée cible : 8–12 minutes
Message unique à retenir :

> « Sur OpenShift AI, on déploie un modèle en natif avec llm-d, puis on le **publie** dans Models-as-a-Service — sans bricolage ExternalModel. »

Prépare l'installation avec `GUIDE-INSTALLATION-DEMO1.md` **avant** d'être en salle.

Avant d'entrer en scène (2 minutes)

Ouvre **deux onglets** déjà connectés :

1. **Dashboard RHOAI**
   `https://rhods-dashboard-redhat-ods-applications.<domaine>`
2. **Terminal** (optionnel, pour 30 secondes de preuve CLI)

Vérifie vite :

```
oc get llminferenceservice,maasmodelref -n rhoai-maas-demo
```

Tu veux voir le modèle `demo-simulator` et le `MaaSModelRef` du même nom.

Fil rouge (à dire une fois)

Dessine mentalement (ou sur un slide) :

```
 [Admin plateforme]
        │
        ▼
LLMInferenceService     ← déploiement natif llm-d
        │
        ▼
MaaSModelRef            ← « publier dans MaaS »
        │
        ▼
Catalogue / gouvernance MaaS
        │
        ▼
(Demo 2) utilisateurs, clés API, quotas
```

Dis clairement : **aujourd'hui = Demo 1 uniquement** (publication).
Les clés API et quotas = **prochaine démo**.

Parcours clics (pas à pas)

Étape 0 — Accueil dashboard (30 s)

1. Affiche la page d'accueil RHOAI.
2. Phrase type :

> « C'est le cockpit OpenShift AI. Tout ce qu'on va montrer est administrable ici, et aussi en GitOps / CLI. »

Ne reste pas sur l'accueil.

Étape 1 — Le projet de démo (1 min)

1. Menu de gauche → **Data Science Projects** (ou **Projets**).
2. Clique sur **`rhoai-maas-demo`**.
3. Phrase type :

> « On isole la démo dans son propre projet. On ne touche pas aux autres modèles déjà présents sur le cluster. »

Si on te demande le modèle GPU legacy :
« Il tourne toujours dans `my-first-model`. On l'a volontairement laissé intact. »

Étape 2 — Montrer le modèle natif (3 min) — moment clé

1. Dans le projet, ouvre la section **Models** / **Model deployments** / déploiements de modèles (le libellé exact peut varier légèrement selon la build 3.5).
2. Sélectionne **`demo-simulator`**.
3. Fais voir que c'est un **`LLMInferenceService`** (chemin **llm-d**), pas un vieux `InferenceService` KServe classique.

Phrase pédagogique (à ne pas rater) :

> « Deux façons d'exposer un modèle à MaaS :
> 1) **natif** : le modèle vit déjà sur le cluster en `LLMInferenceService` — c'est ce qu'on montre ;
> 2) **externe** : on pointe vers un fournisseur cloud — utile, mais ce n'est pas le sujet aujourd'hui. »

4. Si l'UI montre le statut : souligne **Ready** / disponible.

Note honnête si on te demande "c'est quel LLM ?" :

> « Ici on utilise un simulateur CPU llm-d pour démontrer le **mécanisme de publication**, sans monopoliser le GPU unique du sandbox. Le même bouton / le même CR marchent pour un vrai modèle instruct sur GPU. »

Étape 3 — Publication MaaS (3 min) — second moment clé

Deux façons de raconter (choisis **une** selon le temps) :

Option A — Récit "bouton dashboard" (idéal commercial)

1. Montre l'endroit **Publish as MaaS** / publication MaaS dans l'écran du modèle (s'il est visible).
2. Explique :

> « Cocher Publish as MaaS crée un objet `MaaSModelRef` qui enregistre le modèle dans la couche Models-as-a-Service. »

3. Ajoute la transparence :

> « Dans cette sandbox, on l'a déjà créé en déclaratif (CLI/Git), **exactement le même objet** que le bouton aurait créé. »

Option B — Preuve CLI (idéal technique, 20 s)

Dans le terminal déjà ouvert :

```
oc get maasmodelref demo-simulator -n rhoai-maas-demo -o yaml | head -40
```

Montre surtout :

```
spec:
  modelRef:
    kind: LLMInferenceService
    name: demo-simulator
```

> « Même namespace que le modèle. Kind = LLMInferenceService. C'est la preuve que c'est le chemin natif. »

Si le `phase` est `Pending` avec un message du type *Awaiting governance pairing* :

> « Normal en Demo 1 : le backend est sain (`RuntimeReady`). L'accès utilisateurs (abonnement + politique) arrive en Demo 2. »

Étape 4 — Où ça se voit côté MaaS / Gen AI (2 min)

1. Navigue vers l'UI MaaS / Gen AI Studio / catalogue modèles (selon les flags du cluster).
2. Cherche l'entrée liée à **`demo-simulator`** / alias publié.
3. Phrase type :

> « Une fois publié, le modèle entre dans le périmètre gouverné MaaS : c'est à partir de là qu'on pourra abonner des groupes, délivrer des clés, et appliquer des quotas — Demo 2. »

Ne lance **pas** de `curl` avec clé API en Demo 1 (hors scope).

Étape 5 — Clôture (1 min)

Récap en 3 puces à l'oral :

1. **Déployer** en natif avec `LLMInferenceService` (llm-d)
2. **Publier** avec `MaaSModelRef`
3. **Gouverner** l'accès ensuite (Demo 2)

Question de transition :

> « Voulez-vous qu'on enchaîne sur le parcours utilisateur : clé API, quota, révocation ? »

Ce qu'il ne faut pas faire en live

À éviter	Pourquoi
Modifier `my-first-model`	Risque de casser le GPU / une autre démo
Activer `vLLMDeploymentOnMaaS`	Technology Preview — pas le message GA
Promettre un "vrai" LLM 70B sur 1× L4	Capacité insuffisante
Dire "c'est Ready pour les users" sans Demo 2	Faux : il manque subscription + auth policy
Coller un token / secret à l'écran	Sécurité

Aide-mémoire une page (imprime / 2e écran)

```
1. Data Science Projects → rhoai-maas-demo
2. Models → demo-simulator  (= LLMInferenceService natif)
3. Expliquer Publish as MaaS / montrer MaaSModelRef
4. Montrer la présence côté MaaS UI
5. Teaser Demo 2 : clés + quotas
```

Commandes de secours :

```
oc get llminferenceservice -n rhoai-maas-demo
oc get maasmodelref -n rhoai-maas-demo
oc get dsc default-dsc -o jsonpath='{.status.conditions[?(@.type=="ModelsAsAServiceReady
```

FAQ express

Q : C’est quoi la différence avec le modèle déjà installé sur le sandbox ?
R : L’existant est souvent un `InferenceService` “classique”. Ici on montre le chemin **llm-d natif** + publication MaaS.

Q : Pourquoi un simulateur ?
R : 1 GPU déjà pris. On démontre le **processus** sans conflit. Le CR est le même pour un vrai modèle.

Q : Les users peuvent déjà appeler le modèle ?
R : Pas encore de façon gouvernée : il manque la Demo 2 (subscription + auth policy + clés).

Q : On peut le refaire sur un autre sandbox ?
R : Oui — clone du repo GitHub + guide d’installation, scripts idempotents.